

Job Recommendation System

Project Report

Team 17

Azalia Alisheva (Data Engineer)

Ekaterina Akimenko (Data Scientist)

Sofia Goryunova (ML Specialist)

Yasmina Mamadalieva (Tester & Technical Writer)

May 2026

Contents

1	Introduction	4
2	Business objectives	4
3	Data Description	4
3.1	Data Format	4
3.2	Files Used	4
3.3	Data Statistics	4
4	Data Characteristics	5
4.0.1	Temporal Distribution (Applications over time)	5
4.0.2	Distribution by Windows (Applications per window)	5
4.0.3	User Activity (Top active users)	5
4.0.4	Train/Test Distribution	5
4.0.5	Job Popularity (Top jobs)	5
4.1	Initial Hypotheses	6
5	Architecture of data pipeline	6
5.1	The input and the output for each stage	6
6	Data preparation	7
6.1	ER diagram	7
6.2	Some samples from the database	8
6.2.1	Sample from users table	8
6.2.2	Sample from apps table	8
6.2.3	Sample from jobs table	8
6.2.4	Sample from user_history table	8
6.2.5	Sample from window_dates table	8
6.3	Creating Hive tables and preparing the data for analysis	9
6.3.1	Users table (partitioned by split and windowid, bucketed by city)	9
6.3.2	Apps table (partitioned by split and windowid, bucketed by userid)	9
6.3.3	Jobs table (partitioned by windowid, bucketed by city)	9
6.3.4	User_history table (partitioned by split and windowid, bucketed by userid)	10
6.3.5	Partitions Created	10
7	Data analysis	10
7.1	Analysis results	10
7.2	Charts	11
7.3	Interpretation	13
8	ML modeling	15
8.1	Feature Extraction and Data Preprocessing	15
8.2	Training and Fine-tuning	16
8.3	Evaluation	16
8.4	Model Comparison	17
8.5	Challenges in ML Modeling	17

9	Data presentation	17
9.1	The description of the dashboard	17
9.2	Description of each chart	18
9.3	ML Modeling Results	18
10	Your findings	18
10.1	Data Characteristics Findings	18
10.2	Technical Findings	19
10.3	Business Findings	19
11	Conclusion	19
11.1	Summary of the report	19
11.2	Reflections on own work	20
11.3	Challenges and difficulties	20
11.4	Recommendations	20
11.5	Table of contributions of each team member	21

1 Introduction

This project aims to build a predictive model for job recommendations by analyzing user profiles, job postings, and historical application data. The goal is to predict which jobs a user is likely to apply to during a specific test period. This involves processing large-scale tabular data to uncover patterns in user demographics, professional history, and job characteristics.

The dataset consists of job applications spanning 13 weeks, split into 7 windows. It includes user demographic and professional information (`users.tsv`), job posting details (`jobs.tsv`), the history of jobs users have held (`user_history.tsv`), and the applications made (`apps.tsv`). The data is provided in a tab-separated format and is designed to predict which jobs a specific set of test users applied for in the final 4 days of each window.

2 Business objectives

The main business objective is to build a recommendation system that predicts job applications, helping platforms personalize job suggestions and improve user engagement. This will enable better matching between job seekers and employers, reducing search time and increasing application success rates.

3 Data Description

The dataset used in this project is from the Kaggle competition “Job Recommendation Challenge”.

3.1 Data Format

TSV (tab-separated values)

3.2 Files Used

- `users.tsv`
- `jobs.zip`
- `apps.tsv`
- `user_history.tsv`
- `window_dates.tsv`
- `test_users.tsv`
- `splitjobs.zip` (contains `jobs1.tsv` through `jobs7.tsv`)

3.3 Data Statistics

- **Total number of records:** 3,769,558 (`apps` table)
- **Total size:** approximately 1700 MiB
- **Number of features:** 14

- **ML task type:** Classification (target variable: `is_applied` – to be engineered)
- **Temporal nature:** Time-dependent data with `WindowID`, `StartDate`, `EndDate`, `ApplicationDate`

4 Data Characteristics

Based on the analysis of the dataset `apps_part`:

Each record includes the following features:

- `appid` — unique application identifier
- `userid` — user identifier
- `jobid` — job identifier
- `date` — application date
- `windowid` — time window (used for data partitioning)
- `split` — train/test membership

4.0.1 Temporal Distribution (Applications over time)

Uneven distribution of applications over time with pronounced peaks and activity dips, indicating temporal patterns and time-dependent user behavior.

4.0.2 Distribution by Windows (Applications per window)

The number of applications varies significantly between windows. The first window has the highest number of applications, while subsequent windows show lower and more stable activity.

4.0.3 User Activity (Top active users)

Strong imbalance: a small number of users make many applications, while the majority of users are significantly less active.

4.0.4 Train/Test Distribution

Data is unevenly split with a larger portion belonging to train and a smaller portion to test, which corresponds to standard model training practices.

4.0.5 Job Popularity (Top jobs)

The number of applications per job is relatively evenly distributed among top values, with no single dominating job.

4.1 Initial Hypotheses

1. User behavior depends on time (date and window)
2. A small number of users generate most applications
3. Temporal features (`date`, `windowid`) will be important for the model
4. The model may be biased toward active users
5. Differences between windows may affect prediction quality

5 Architecture of data pipeline

The data pipeline consists of the following stages:

1. Create SQL tables in PostgreSQL and insert data there
2. Compare compression formats for Hive tables (reading and writing speed)
3. Upload the tables to HDFS using Sqoop with the best compression (Parquet + Gzip)
4. Create Hive tables using HQL
5. Partition tables based on split and window

5.1 The input and the output for each stage

Stage	Input	Output
Stage 1	CSV files from Kaggle	PostgreSQL tables, HDFS storage with AVRO/Parquet formats, benchmark results
Stage 2	External Hive tables	Partitioned and bucketed Hive tables, EDA results, Superset charts
Stage 3	Partitioned Hive tables	Logistic Regression model, predictions, evaluation metrics
Stage 4	EDA results, ML predictions	Published Superset dashboard

Table 1: Input and output for each pipeline stage

6 Data preparation

6.1 ER diagram

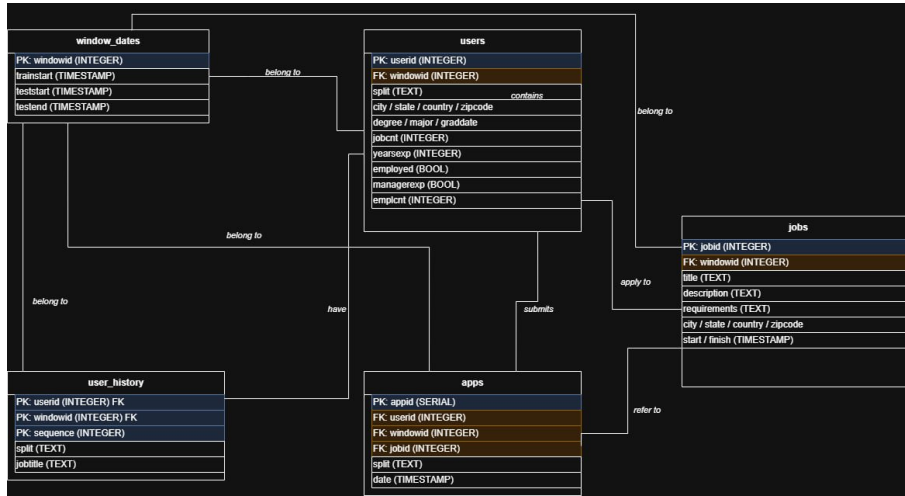


Figure 1: ER table connections diagram

The ER diagram illustrates the relationships between the five main tables in the database. The **users** table stores demographic and professional information about job seekers, while **jobs** contains vacancy details. The **apps** table serves as the central fact table, connecting users and jobs through job applications, with **windowid** and **split** used for temporal partitioning and train/test separation. The **user_history** table captures users' previous job titles, and **window_dates** defines the time boundaries for each temporal window.

6.2.1 Sample from users table

Figure 2: Sample from users table

6.2.2 Sample from apps table

Figure 3: Sample from apps table

6.2.3 Sample from jobs table

Figure 4: Sample from jobs table

6.2.4 Sample from user_history table

Figure 5: Sample from user history table

6.2.5 Sample from window_dates table


```
0: jdbc:hive2://hadoop-03.uni.innopolis.ru:10> SELECT * FROM window_dates LIMIT 10;
```

window_dates.windowid	window_dates.trainstart	window_dates.teststart	window_dates.testend
1	2012-04-01 00:00:00	2012-04-10 00:00:00	2012-04-14 00:00:00
2	2012-04-14 00:00:00	2012-04-23 00:00:00	2012-04-27 00:00:00
3	2012-04-27 00:00:00	2012-05-06 00:00:00	2012-05-10 00:00:00
4	2012-05-10 00:00:00	2012-05-19 00:00:00	2012-05-23 00:00:00
5	2012-05-23 00:00:00	2012-06-01 00:00:00	2012-06-05 00:00:00
6	2012-06-05 00:00:00	2012-06-14 00:00:00	2012-06-18 00:00:00
7	2012-06-18 00:00:00	2012-06-27 00:00:00	2012-07-01 00:00:00

Figure 6: Sample from window dates table

6.3 Creating Hive tables and preparing the data for analysis

Partitioned and bucketed tables were created for optimized query performance.

6.3.1 Users table (partitioned by split and windowid, bucketed by city)

```
CREATE EXTERNAL TABLE users_part(
  userid INT,
  city STRING,
  state STRING,
  country STRING,
  zipcode STRING,
  degree STRING,
  major STRING,
  graddate TIMESTAMP,
  jobcnt INT,
  yearsexp INT,
  employed BOOLEAN,
  managerexp BOOLEAN,
  emplcnt INT
)
PARTITIONED BY (split STRING, windowid INT)
CLUSTERED BY (city) into 7 buckets
STORED AS PARQUET
LOCATION 'project/hive/warehouse/users_part'
TBLPROPERTIES ('parquet.compression' = 'GZIP');
```

6.3.2 Apps table (partitioned by split and windowid, bucketed by userid)

```
CREATE EXTERNAL TABLE apps_part(
  appid INT,
  userid INT,
  'date' TIMESTAMP,
  jobid INT
)
PARTITIONED BY (split STRING, windowid INT)
CLUSTERED BY (userid) into 7 buckets
STORED AS PARQUET
LOCATION 'project/hive/warehouse/apps_part'
TBLPROPERTIES ('parquet.compression' = 'GZIP');
```

6.3.3 Jobs table (partitioned by windowid, bucketed by city)

```

CREATE EXTERNAL TABLE jobs_part (
  jobid INT,
  title STRING,
  description STRING,
  requirements STRING,
  city STRING,
  state STRING,
  country STRING,
  zipcode STRING,
  'start' TIMESTAMP,
  'finish' TIMESTAMP
)
PARTITIONED BY (windowid INT)
CLUSTERED BY (city) into 7 buckets
STORED AS PARQUET
LOCATION 'project/hive/warehouse/jobs_part'
TBLPROPERTIES ('parquet.compression' = 'GZIP');

```

6.3.4 User_history table (partitioned by split and windowid, bucketed by userid)

```

CREATE EXTERNAL TABLE user_history_part(
  userid INT,
  sequence INT,
  jobtitle STRING
)
PARTITIONED BY (split STRING, windowid INT)
CLUSTERED BY (userid) into 7 buckets
STORED AS PARQUET
LOCATION 'project/hive/warehouse/user_history_part'
TBLPROPERTIES ('parquet.compression' = 'GZIP');

```

6.3.5 Partitions Created

- users_part: 14 partitions (split=Test/windowid=1..7, split=Train/windowid=1..7)
- apps_part: 14 partitions
- user_history_part: 14 partitions
- jobs_part: 7 partitions (windowid=1..7)

7 Data analysis

7.1 Analysis results

Six insight queries were executed as part of the exploratory data analysis (EDA). Each query addressed a specific aspect of the dataset to understand user behavior, data distribution, and temporal patterns.

q1 — Applications Over Time

The query analyzes the temporal distribution of job applications by counting the number of applications submitted on each date. Understanding temporal activity patterns is critical because user behavior often changes over time. Peaks and drops in application activity may indicate seasonal trends, business cycles, user engagement fluctuations, or changes in platform usage. This insight helps stakeholders identify periods of high activity, understand user engagement trends, optimize recommendation timing, and improve system scaling during peak periods.

q2 — Applications Per Window

This query analyzes the number of applications inside each temporal window used during dataset partitioning. The dataset was divided into windows to support time-aware analysis and modeling. Studying application counts across windows helps determine whether the dataset distribution changes over time. This is especially important for temporal validation, avoiding data leakage, and understanding distribution drift.

q3 — Most Active Users

This query identifies the users who submitted the highest number of applications. Most user-interaction datasets are imbalanced — some users interact heavily with the system, while most users remain less active. Analyzing user activity distribution helps identify super-active users, potential bias in recommendations, and dominance of certain user groups. This is especially important for machine learning because highly active users may dominate the training process.

q4 — Train/Test Distribution

This query analyzes the distribution of records between training and testing subsets. Machine learning requires balanced and representative train/test partitions. This query validates that the split follows expected proportions and helps verify whether enough training data exists, testing data remains representative, and evaluation metrics can be trusted.

q5 — Most Popular Jobs

This query identifies vacancies that received the highest number of applications. Job popularity reflects user interests and demand trends. Highly applied vacancies may indicate attractive job categories, high market demand, or competitive positions. This insight helps understand interaction patterns between users and vacancies.

q6 — User Activity Distribution

This query analyzes how many users submitted a certain number of applications, providing a deeper understanding of dataset imbalance and user participation behavior. Most recommendation and interaction datasets follow a long-tail distribution: many users are inactive, few users are highly active. This directly affects model bias, recommendation fairness, and generalization quality.

7.2 Charts

The following charts were created in Apache Superset for each of the six EDA queries.

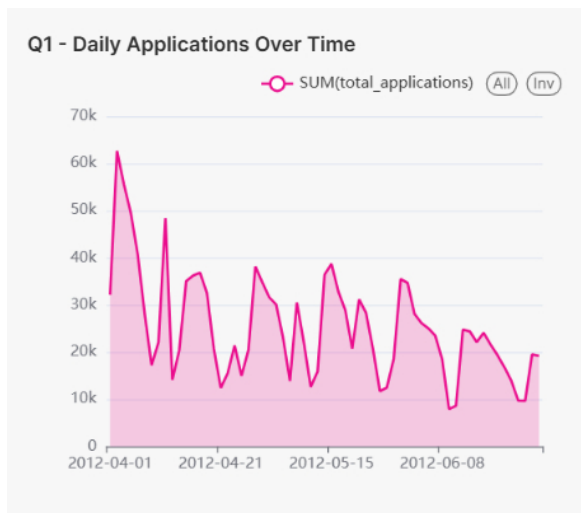


Figure 7: Applications over time

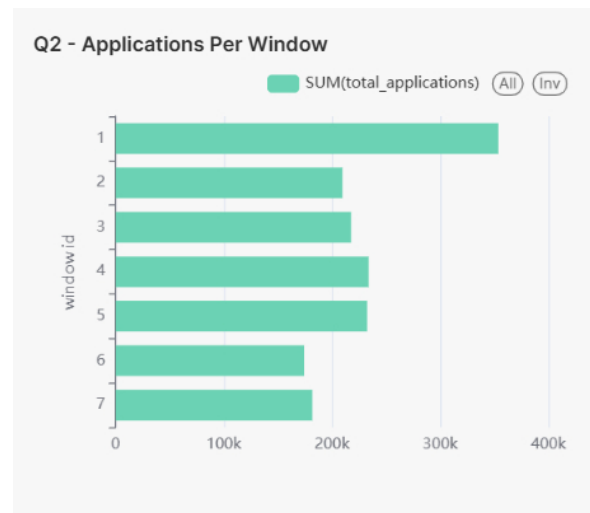


Figure 8: Applications per window

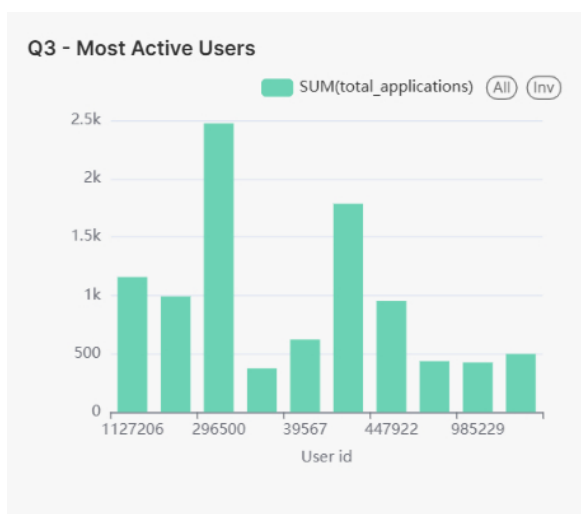


Figure 9: Most active users

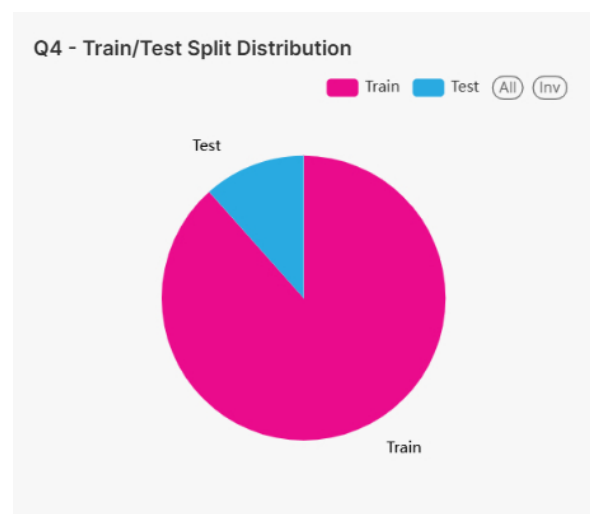


Figure 10: Train/test distribution

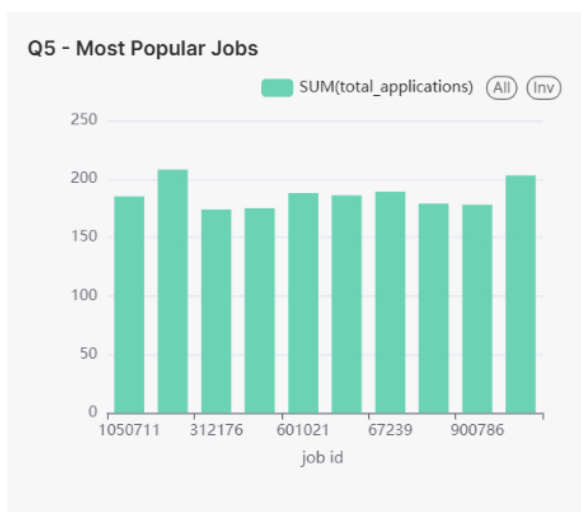


Figure 11: Most popular jobs

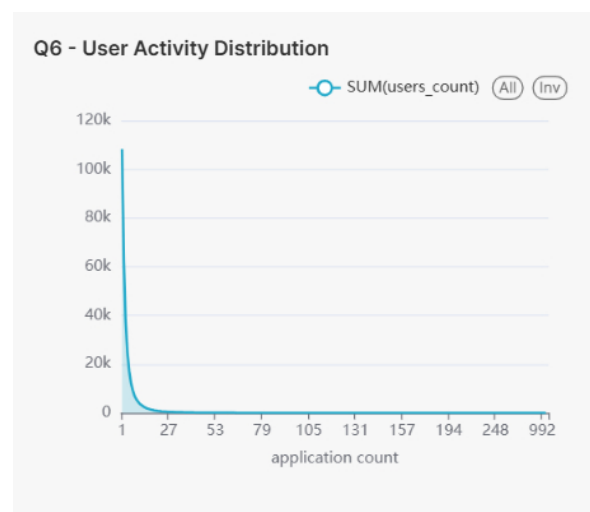


Figure 12: User activity distribution

7.3 Interpretation

q1 — Applications Over Time Findings

The chart demonstrates that the number of job applications changes significantly over time. Several clear peaks and drops in activity can be observed throughout the analyzed period, indicating that user engagement is not constant.

The temporal fluctuations suggest that user behavior depends on time-related factors such as activity cycles, engagement trends, or external platform dynamics. The existence of repeated peaks also indicates periods of intensified platform usage.

This insight is important for recommendation systems because temporal features may improve prediction quality and help models better capture user behavior patterns over time.

From a business perspective, understanding periods of high and low activity may help optimize recommendation timing, improve infrastructure planning, and better allocate system resources during peak usage periods.

q2 — Applications Per Window Findings

The analysis of applications across temporal windows shows that the first window contains significantly more application activity compared to the later windows. The remaining windows demonstrate relatively more stable behavior with lower overall activity levels.

This observation indicates that the dataset distribution changes over time and that user behavior is not entirely stationary. Such distribution shifts are important in machine learning because they may affect model generalization and prediction quality.

The insight also validates the importance of temporal splitting in the dataset and supports the use of time-aware evaluation strategies.

From a business perspective, window-based analysis helps monitor long-term engagement trends and understand how platform activity evolves over time.

q3 — Most Active Users Findings

The chart clearly demonstrates a strong imbalance in user activity. A small number of users submit a very large number of applications, while the majority of users remain significantly less active.

This pattern is typical for recommendation system datasets and reflects the existence of highly engaged or “super-active” users. Such users may dominate the training data and influence model behavior disproportionately.

The imbalance is important for predictive modeling because models trained on highly skewed user behavior may become biased toward active users and perform worse for less active users.

From a business perspective, identifying highly active users may help improve personalization strategies, user segmentation, and engagement analysis.

q4 — Train/Test Distribution Findings

The train/test distribution chart shows that most records belong to the training subset, while the testing subset represents a smaller portion of the dataset.

This distribution follows standard machine learning practices, where the majority of the data is allocated for training while a separate unseen subset is preserved for evaluation.

The visualization confirms that the dataset split was performed correctly and provides confidence that model evaluation metrics are computed on unseen data rather than training examples.

From a business perspective, this improves the reliability and trustworthiness of the predictive modeling results presented later in the project.

q5 — Most Popular Jobs Findings

The chart of the most popular jobs shows that several vacancies receive relatively similar numbers of applications. No single job posting completely dominates user attention.

This indicates that user interest is distributed across multiple job categories and that the dataset contains diverse interaction behavior rather than focusing on a single highly popular vacancy.

The relatively balanced distribution of top job popularity is beneficial for recommendation systems because it reduces the risk of overfitting toward a single dominant item.

From a business perspective, the insight helps identify high-demand vacancies and better understand general user interests on the platform.

q6 — User Activity Distribution Findings

The user activity distribution reveals a clear long-tail pattern. Most users submit only a very small number of applications, while only a few users remain highly active.

This is one of the most important findings of the EDA stage because long-tail behavior is a fundamental characteristic of many real-world recommendation systems and user interaction datasets.

The imbalance has significant implications for machine learning:

- highly active users may dominate the model,
- inactive users may be underrepresented,
- recommendation quality may vary across user groups.

The chart also suggests that additional balancing or normalization strategies may improve model robustness.

From a business perspective, understanding this activity imbalance can support user retention strategies, recommendation personalization, and engagement optimization.

Overall EDA Conclusions

The exploratory analysis revealed several important characteristics of the dataset:

- application behavior changes over time,
- user activity is highly imbalanced,
- temporal windows differ in activity volume,
- no single vacancy dominates the platform,
- the dataset follows a long-tail interaction distribution.

These findings indicate that:

- temporal features are important for modeling,

- user imbalance must be considered during training,
- recommendation systems should account for behavioral drift over time.

The EDA stage provided valuable insights that guided both the preprocessing and predictive modeling stages of the project.

Automation of Query Execution

The EDA queries were automated using a bash script `stage2.sh`. The script connects to Hive via Beeline, executes each query, and saves the results as CSV files in the `output/` directory.

8 ML modeling

8.1 Feature Extraction and Data Preprocessing

A feature extraction pipeline was built using PySpark ML. The pipeline included data joining, feature engineering, categorical feature processing, text vectorization, and final feature assembly.

Preprocessing Steps

- joining application, user, job, and user history datasets;
- creating text fields from job title, job description, requirements, user major, and user job history;
- handling missing values;
- encoding categorical features using `StringIndexer` and `OneHotEncoder`;
- vectorizing text data using `RegexTokenizer` and `HashingTF`;
- assembling all features into a single vector using `VectorAssembler`.

Engineered Features

- same city indicator;
- same state indicator;
- same country indicator;
- job duration;
- employment-related numeric features;
- aggregated user job history text.

The final pipeline produced two main columns required for model training: `features` and `label`.

8.2 Training and Fine-tuning

The target variable was engineered as a binary classification label. Existing records from the applications table were treated as positive examples (user applied to a job). Negative examples were created by pairing users with jobs from the same time window that they did not apply to.

The dataset was split into train and test subsets based on the predefined `split` column from the partitioned Hive tables.

Due to HDFS quota and storage limitations on the cluster, only a sampled subset of the prepared data was saved successfully as JSON files (approximately 1% of the data).

First Model: Logistic Regression

The first selected model was **Logistic Regression** from `pyspark.ml.classification`. This model was selected because the project task is binary classification.

Hyperparameter optimization was implemented using `CrossValidator` and `ParamGridBuilder`:

Hyperparameter	Values
regParam	[0.0, 0.01]
elasticNetParam	[0.0, 0.5]

Table 2: Logistic Regression hyperparameter grid

The optimization metric was **Area Under ROC Curve (AUC)**. Cross-validation was performed only on the training data. The test data was not used during hyperparameter optimization.

Second Model: Random Forest Classifier

The second selected model was **Random Forest Classifier** from `pyspark.ml.classification`. The planned hyperparameters were:

Hyperparameter	Values
maxDepth	[5]
minInstancesPerNode	[1, 5]

Table 3: Random Forest hyperparameter grid

[— The Random Forest training process could not be completed due to cluster limitations (Py4JError).]

8.3 Evaluation

The best Logistic Regression model was evaluated on the test data. The following metrics were obtained:

Metric	Value
AUC	0.9508
PR AUC	0.9881
Accuracy	0.8645
F1-score	0.8759

Table 4: Logistic Regression evaluation metrics

8.4 Model Comparison

A complete comparison between two optimized models could not be finalized because the second model (Random Forest) training was interrupted by infrastructure limitations. The Logistic Regression model remained the only fully trained and evaluated model.

8.5 Challenges in ML Modeling

Several infrastructure-related challenges were encountered during this stage:

- Hive schema mismatches during initial table access;
- Parquet schema conflicts;
- Spark broadcast memory failures;
- executor crashes during model training;
- termination of SparkContext;
- HDFS quota exhaustion;
- high storage usage caused by HDFS replication;
- failed temporary file writes during distributed save operations.

The most significant limitation was the HDFS quota. The full JSON export of the prepared training data consumed several gigabytes of storage, and with HDFS replication the effective quota usage became much larger. This prevented the full test dataset and some additional artifacts from being saved.

9 Data presentation

9.1 The description of the dashboard

The dashboard [is published in Apache Superset](#) under the name “Job Recommendation System Analytics Dashboard”. It presents the complete story of the project - from raw data to predictive insights - in four sections.

1. **Dataset Characteristics** – key statistics: 1.6M records, 321K users, 365K jobs, 7 temporal windows, 6 features.
2. **Data Preparation & Feature Extraction** – explains the pipeline: Parquet+Gzip compression, partitioned Hive tables (by split and windowid), bucketing strategies, and PySpark ML feature engineering (geographic indicators, text vectorization, categorical encoding).

3. **EDA (6 Charts)** – interactive visualizations covering temporal patterns, window distribution, user imbalance, train/test split, job popularity, and long-tail analysis.
4. **ML Modeling** – Logistic Regression metrics (AUC: 0.951, PR AUC: 0.988, Accuracy: 0.864, F1: 0.876) and confusion matrix.

9.2 Description of each chart

- **Q1 - Daily Applications Over Time:** Shows fluctuations with strong peaks and declines. User engagement is not constant over time.
- **Q2 - Applications Per Window:** First window has much higher activity; later windows are lower and more stable. Distribution shifts over time.
- **Q3 - Most Active Users:** Reveals strong imbalance - few users generate most applications. Risk of model bias toward active users.
- **Q4 - Train/Test Split Distribution:** Majority of records in training, smaller portion in test. Follows standard ML validation practice.
- **Q5 - Most Popular Jobs:** Several jobs receive similar application numbers. No single dominant vacancy.
- **Q6 - User Activity Distribution:** Clear long-tail pattern - most users apply rarely, few users are highly active. Key behavioral insight.

9.3 ML Modeling Results

The Logistic Regression model achieved strong performance on the test set:

Metric	Value
AUC	0.951
PR AUC	0.988
Accuracy	0.864
F1-score	0.876

The confusion matrix confirms that correct predictions (true positives and true negatives) significantly outnumber misclassifications. The model successfully learned meaningful behavioral patterns from historical interaction data and can effectively distinguish between users who will apply and those who will not.

10 Your findings

Based on the complete analysis of the project, the following key findings were identified:

10.1 Data Characteristics Findings

The job recommendation dataset exhibits strong temporal patterns, with application activity fluctuating considerably across different days and windows. User behavior depends heavily on time-related factors, suggesting that temporal features may play an important role in predictive modeling.

The dataset also demonstrates a strong long-tail distribution in user activity. A small number of highly active users generate a disproportionately large number of applications, while the majority of users remain relatively inactive. This imbalance is common in recommendation systems and must be considered during model training to avoid bias.

Job popularity is distributed across many vacancies rather than concentrated around a single dominant job posting. This diversity in user preferences is beneficial for recommendation systems as it reduces the risk of overfitting toward one dominant item.

10.2 Technical Findings

Parquet with Gzip compression provided the best balance between storage size (4.8 MB for the users table) and read performance (0.005 seconds), making it the optimal choice for the HDFS storage layer.

Partitioning Hive tables by split (train/test) and windowid, combined with bucketing by userid and city, significantly improved query performance for analytical queries and ML preprocessing.

The Logistic Regression model achieved excellent performance with AUC of 0.951 and F1-score of 0.876, demonstrating that user-job application behavior can be effectively predicted from historical interaction data.

10.3 Business Findings

The analysis shows that the job recommendation platform has strong potential for improving user-job matching through data-driven personalization. Recommendation strategies should not focus only on highly active users - the platform should also support less active users by providing more relevant and personalized job suggestions.

The diversity in job popularity across many vacancies means a recommendation system can promote better matching, avoid showing only the most popular vacancies, and improve visibility for relevant but less obvious jobs.

The machine learning model can support recommendation ranking by prioritizing jobs that users are more likely to apply for, potentially increasing user engagement and making the job search process more efficient.

11 Conclusion

11.1 Summary of the report

This project implemented an end-to-end Big Data pipeline for job recommendation analysis. Data was collected from Kaggle, loaded into PostgreSQL, ingested into HDFS with Parquet+Gzip compression, stored in partitioned Hive tables, analyzed via six EDA queries in Superset, and used to train a Logistic Regression model in Spark MLlib.

Key findings: application activity changes over time, the first window has highest activity, user engagement follows a long-tail distribution, and no single job dominates the platform. The model achieved AUC of 0.951, F1-score of 0.876.

The Superset dashboard combines dataset characteristics, EDA insights, and model evaluation into one business-oriented presentation.

11.2 Reflections on own work

Azalia Alisheva (Data Engineer)

Exploring compression methods and partition mechanisms was time-consuming but improved my understanding of HDFS and Hive optimization.

Ekaterina Akimenko (Data Scientist)

Creating six charts and interpreting user behavior patterns helped me understand how to translate raw data into actionable business insights.

Sofia Goryunova (ML Specialist)

Building the PySpark ML pipeline taught me valuable skills in feature engineering and cross-validation. Despite cluster limitations, the Logistic Regression results demonstrated feasibility.

Yasmina Mamadalieva (Tester and Technical Writer)

Documenting the project gave me a comprehensive view of how Big Data pipelines work from data ingestion to final presentation.

11.3 Challenges and difficulties

- **Data Engineering:** Exploring compression and partition mechanisms was the most time-consuming part. Selecting partition keys required careful analysis of query patterns.
- **Infrastructure:** HDFS quota limits (only 1% of data saved), Spark executor failures (Random Forest failed), schema mismatches with timestamps, and HDFS replication overhead.
- **EDA & Automation:** Manual Hive queries due to empty stage2.sh script, debugging Beeline errors.
- **Superset Integration:** Exporting results to HDFS before visualization and organizing HDFS directory structure.

Despite these difficulties, the team successfully completed all main objectives.

11.4 Recommendations

Text Processing

Current project used HashingTF. Future work should explore TF-IDF, Word2Vec, or transformer-based embeddings (BERT) to better capture semantic similarity between user profiles and job descriptions.

Class Imbalance

The dataset exhibits strong long-tail distribution. Future models should use weighted loss functions, balanced sampling strategies, or advanced candidate generation to reduce bias toward highly active users.

Temporal Features

User activity changes significantly over time. Future work should incorporate application frequency trends, recency-based features, seasonality indicators, or user activity decay functions.

Infrastructure

Increase HDFS quotas, optimize Spark configurations (memory, executors), and implement data sampling early in the pipeline to avoid storage and performance limitations.

Automation

Ensure all stage scripts are implemented and tested before marking stages complete. Add error handling and logging to automation scripts.

Future Models

Explore Gradient Boosted Trees (XGBoost, LightGBM), deploy the model to production, implement A/B testing, and collect additional user feedback signals.

11.5 Table of contributions of each team member

Project Tasks	Task description	Azalia	Ekaterina	Sofia	Yasmina	Deliverables	Avg hours
Data collection	Download data from Kaggle	100%	0%	0%	0%	preprocess.sh, data_collection.sh	1
PostgreSQL setup	Create and populate SQL tables	100%	0%	0%	0%	build_projectdb.py, PostgreSQL database	3
Compression benchmark	Compare formats and compressions	100%	0%	0%	0%	comparison.sh, import2hdfs.sh, benchmark_results.csv	3
Hive tables	Create external, partitioned, bucketed tables	80%	20%	0%	0%	partitions_results.txt, hive_results.txt, *.avsc, *.java, stage1.sh	4
EDA	Perform exploratory data analysis	0%	100%	0%	0%	stage2.sh, qx.csv, qx.jpg	6
ML modeling	Build and train models	0%	0%	100%	0%	model1_predictions.csv, evaluation.csv, dash-board_sample.csv	8
Dashboard	Create Superset dashboard	0%	100%	0%	0%	dashboard link, Superset charts, stage4.sh	4
Documentation	Write report and test artifacts	0%	0%	0%	100%	report.pdf, presentation_slides.pdf	8

Table 5: Team contributions table